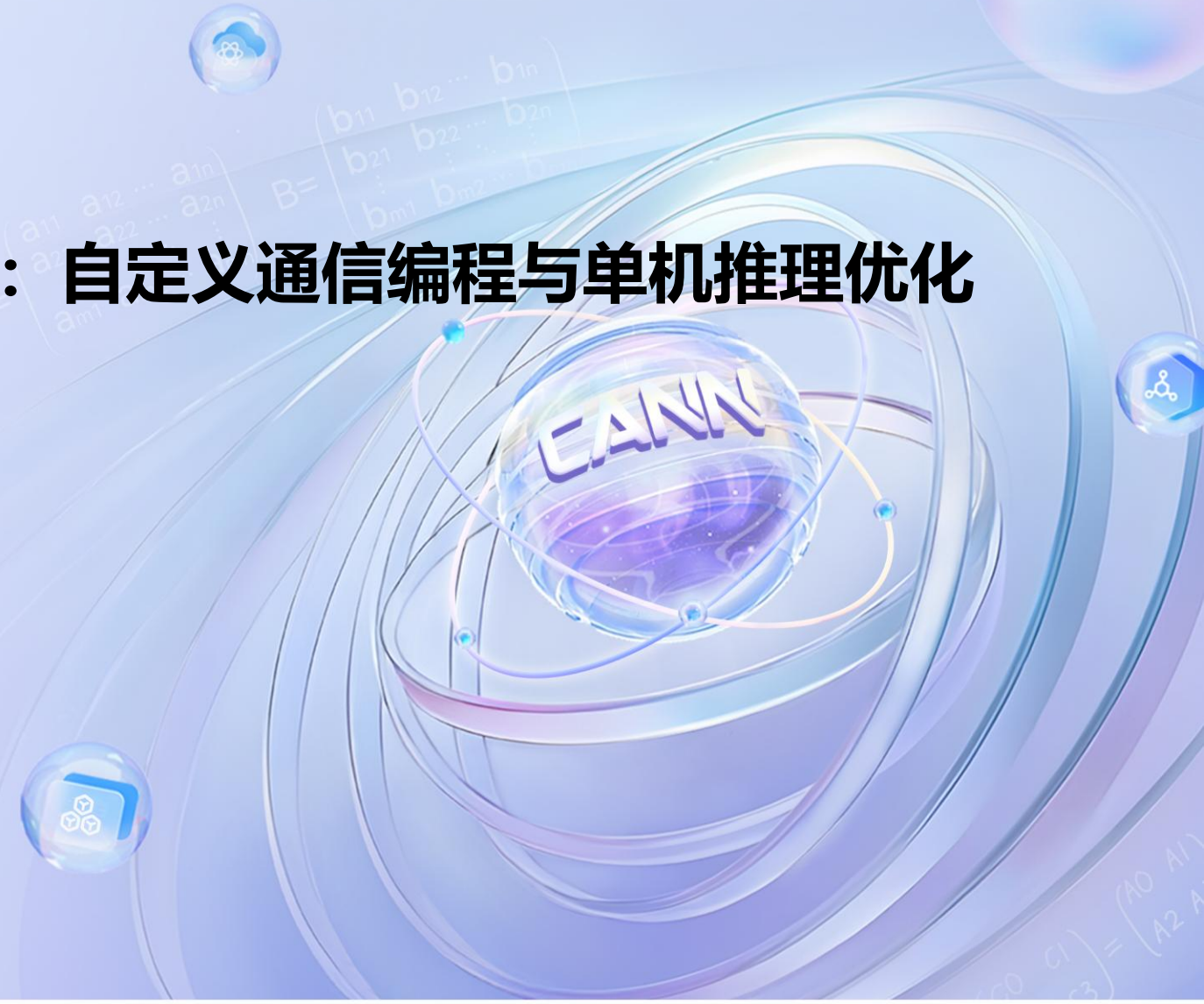
Ascend 950极低时延加速技巧：自定义通信编程与单机推理优化

——李健、虎鲸、君见



CANN

目录

Part 1 自定义通信编程架构及API介绍

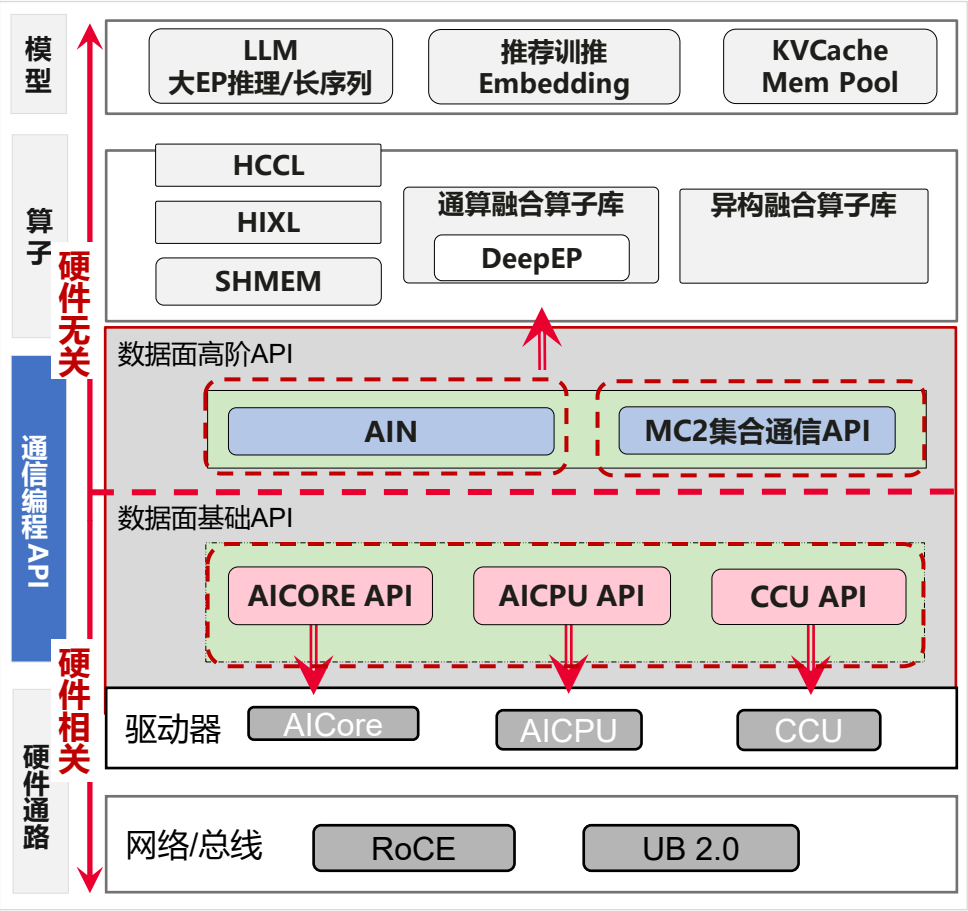
Part 2 基于通信API的高性能算子实践

Part 3 单机极低时延推理参考实践

ASC-COMM：驱动3种通信引擎，释放芯片潜能，支撑通算融合算子高性能编程

ASC-COMM是CANN提供的高性能自定义通信编程库，面向集群场景，屏蔽通信通路，支撑通信算子与通算融合算子高效编程

- **多种驱动方式：**提供AICORE直驱、代理（AICPU、CCU）等完善的高性能通信编程API
- **极简适配：**极简API，屏蔽昇腾芯片代际差异，支持多种网络与灵衢互联（RoCE/UB等），支持通信资源自动检索与链路智能优选
- **多层次开放：**开放不同层级，不同引擎的高性能通信API



提供高性能，多引擎支持的Ascend通信编程底座

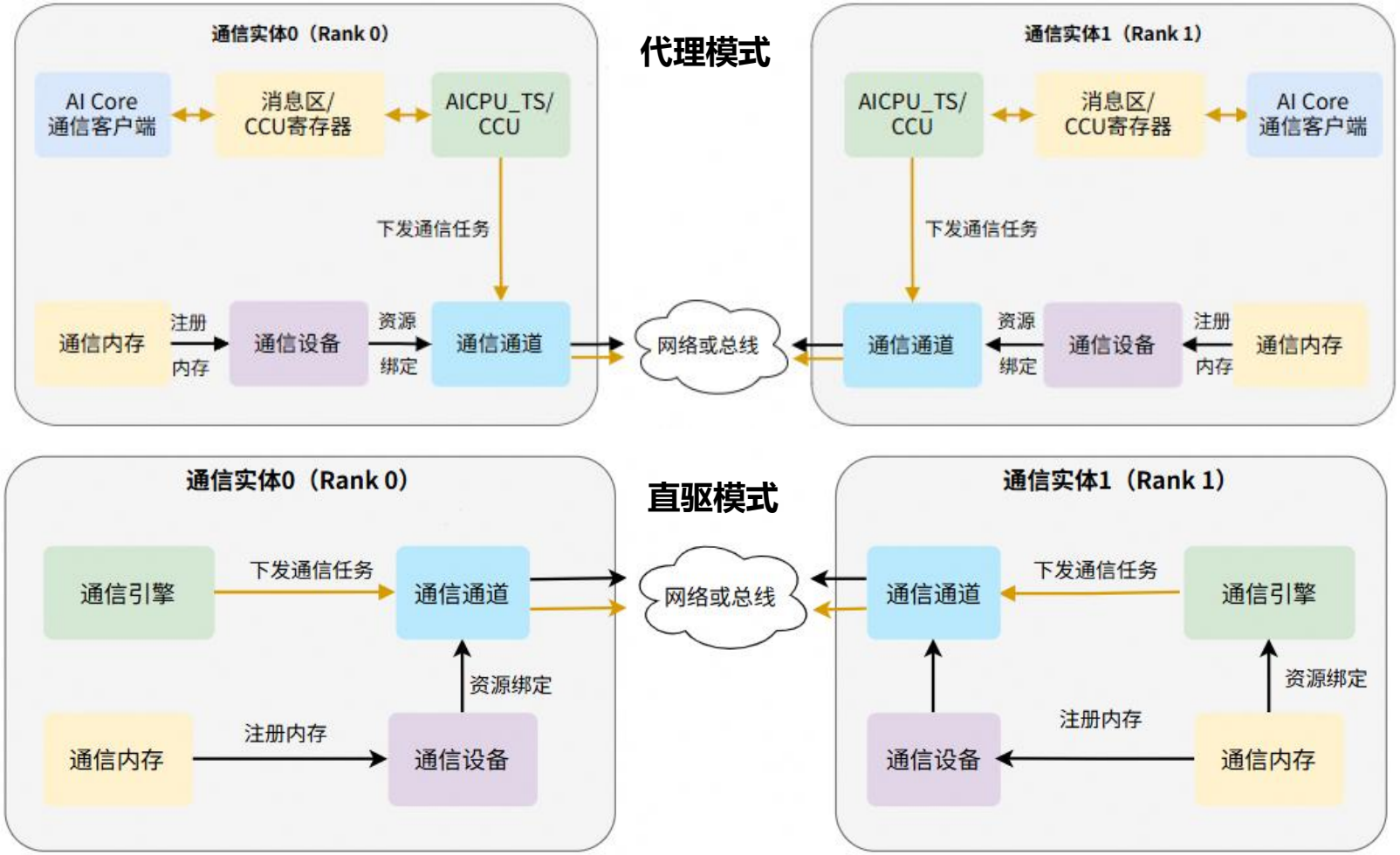
1. **MC2 集合通信API：**
   支撑MC2算子，覆盖5种集合通信原语API和相关通信算法，MC2技术的通信底座。
2. **AIN API：**
   是基于对称内存通信的API，AIN可以提供极简的单边通信直驱能力。
3. **通信数据面基础API：**
   直接封装芯片指令的底层通信API，支撑集合通信和MC2等通信和通算融合业务。支持极致灵活性需求。

通信引擎	执行主体	主要功能
AIV	Vector Core	执行通信Kernel中的算法逻辑，以及数据处理、搬运和归约等操作。
AI CPU+TS	AI CPU+TS	采用AI CPU+TS执行模式。其中STARS是Device侧的任务调度器，负责调度AI CPU Kernel以及通信任务。AI CPU Kernel由STARS调度执行后，会向任务队列提交数据搬运等通信任务，最后再由STARS调度到相应的执行器。
CCU（集合通信加速单元）	集合通信专用硬件引擎	执行预置通信指令，并使用内部CCU Buffer完成数据暂存、搬运、同步和片上归约等操作。

ASC-COMM：驱动3种通信引擎，释放芯片潜能，支撑通算融合算子高性能编程

自定义通信编程库支持两种通信模型：直驱模式和代理模式

- **代理模式**：该模式下通过AICore客户端发起通信请求，AICPU_TS/CCU服务端执行通信任务。
- **直驱模式**：该模式下通过通信引擎直接向底层链路发起并直接执行通信任务，



代理模式

- **AICPU_TS 服务端**

AI Core客户端通过消息区写入通信请求和任务参数，并通知AICPU_TS处理；AICPU_TS服务端读取请求后操作Channel完成本端与对端之间的数据传输，再通过消息区返回完成状态。

- **CCU 服务端**

AI Core客户端通过通用寄存器写入通信请求和任务参数，并通过同步寄存器通知CCU处理；CCU服务端读取请求后操作Channel完成本端与对端之间的数据传输，再更新同步寄存器中的完成状态。

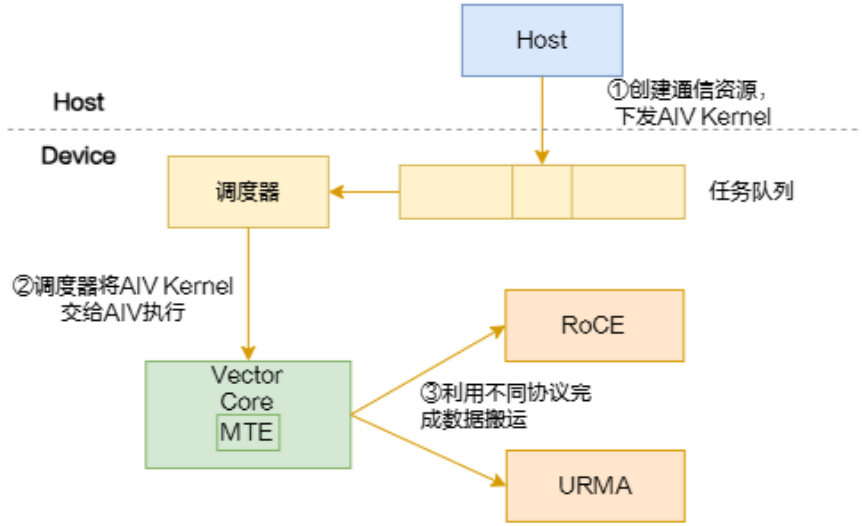
直驱模式

通信引擎直接执行用户下发的指令，将要执行的通信任务描述记录在Channel中，包括通信内存地址、通信操作等信息，根据不同协议驱动硬件执行。

ASC-COMM：自定义通信编程API全景

代码仓路径：https://gitcode.com/cann/asc-comm/tree/master/include





AIV直驱模式

- **AIN单边通信API**

基于对称内存的单边通信库。AIN通过目标成员、窗口和偏移发起Put、Get或Signal等任务，该模式可以很大程度上简化集群编程

- **Hcomm层API**

Hcomm层进行点对点任务，通过Channel确定通信对端，并将本地地址和远端地址分别作为数据的来源和目标。该模式适合目的地址动态变化、Buffer布局不一致以及需要精细控制的场景。

- **Jetty层API**

Jetty层提供底层URMA通信原语，提供最灵活、高性能的通信控制。该模式与底层原语抽象更为贴切。

目录

Part 1 自定义通信编程架构及API介绍

Part 2 基于通信API的高性能算子实践

Part 3 单机极低时延推理参考实践

算子背景介绍

MoE 的专家并行（Expert Parallel，EP）

- 专家分布在不同 rank 上（Expert Parallel，EP）
- 每个 token 选出 Top-k 个专家，由这Top-k个专家进行FFN计算处理
- 同一 token 选的专家可能落在不同 rank或相同rank
- 输入需要跨卡送到目标专家；专家输出还要回到 token 原始 rank

两个原语

Dispatch • 分发

按 Top-k 路由把 token 送到目标专家所在 rank，整理成“按专家连续排列”的输入，供GMM按专家计算。

Combine • 聚合

把专家输出送回 token 来源 rank，按原始 token 顺序、按 Top-k 槽位聚合结果。

Dispatch&Combine的两个阶段

Dispatch



Combine

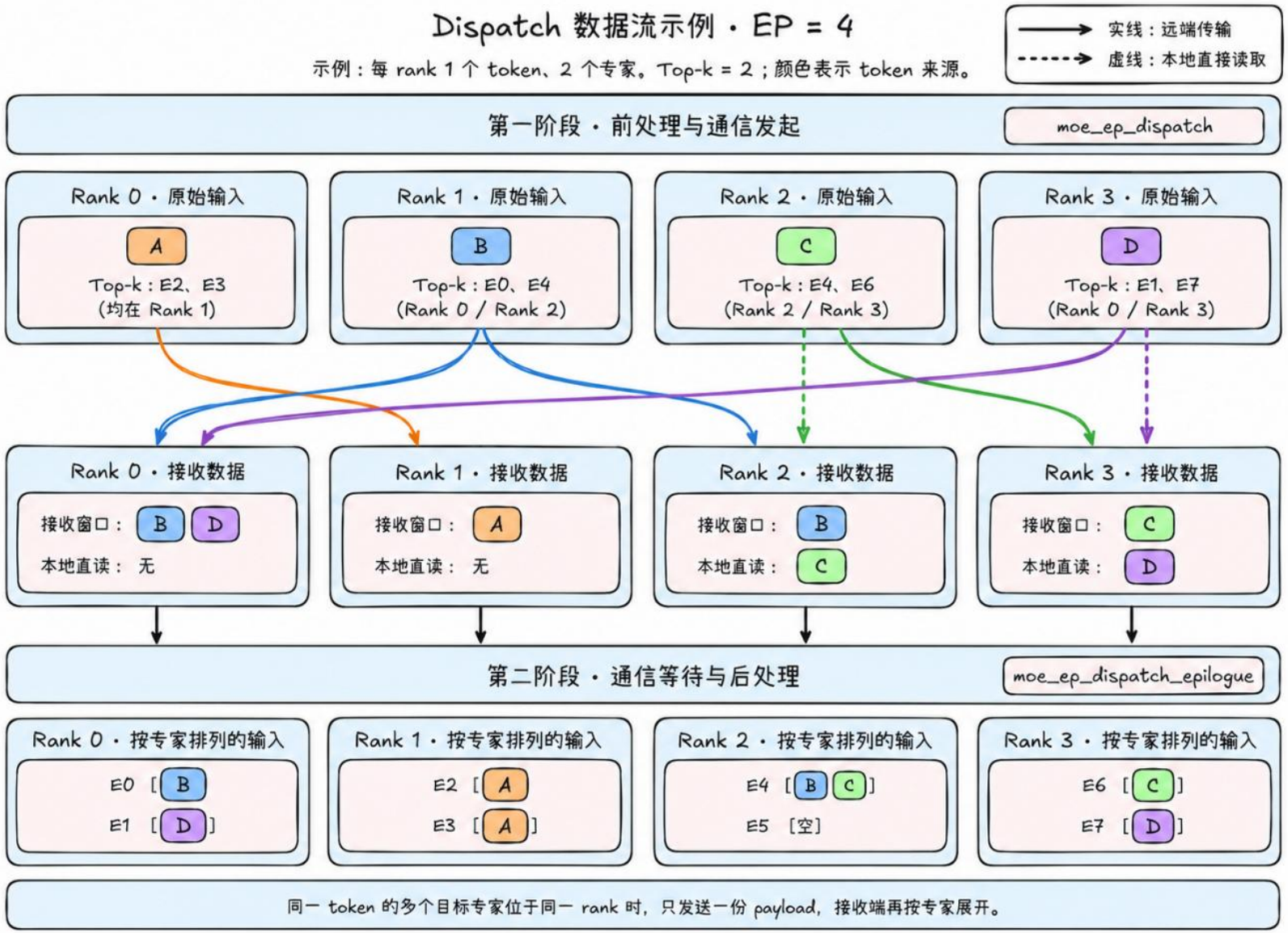


· 三段按块流水、互相重叠；URMA 由网卡引擎代办，通信期间 AIV 空出做 local 处理（dual-pipe）

· combine 先做本地规约再回传，通信量随本卡专家数成比例下降

操作	第一阶段：前处理与通信发起	第二阶段：通信等待与后处理
Dispatch	统计路由、交换计数、准备元数据并发起 token 发送	等待接收完成，按专家展开并输出 token 和回传所需的元数据
Combine	根据回传元数据计算目标位置，发起专家输出发送	等待接收完成，按原始 token 聚合专家输出

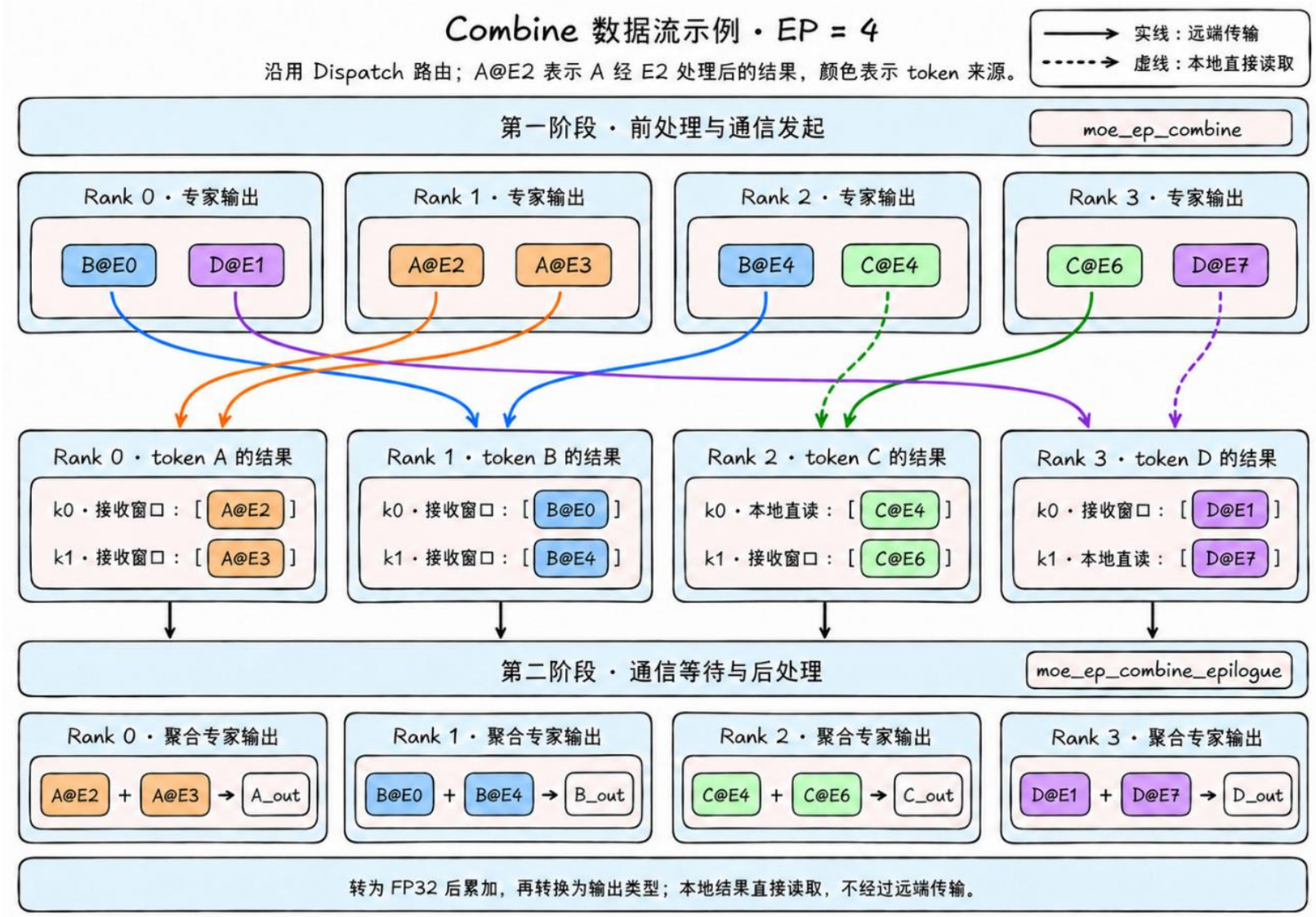
Dispatch数据流图示



图中省略计数交换、通知和元数据的具体布局，重点表示 payload 的流向：

- Token A 同时命中 Rank 1 上的 两个专家E2、E3，因此只向 Rank 1 发送一份 payload，再在通信等待与后处理阶段展开成两份专家输入；
- Token B命中rank0的E0，rank2的E4，正常发送给rank0和rank2；
- Token C→E4、Token D→E7 为本地路径，token 直接从本 rank 的输入读取。

Combine数据流图示



Combine 沿用上图的路由

- A@E2 表示 token A 经专家 E2 处理后的结果，其他标记同理。
- Token A 的两份专家结果分别回到 Rank 0 的两个 Top-k 槽位，再聚合成 A 的输出；B、C、D 同理。
- 图中的本地结果槽位仅表示逻辑对应关系，实际由通信等待与后处理算子直接读取本地专家输出。

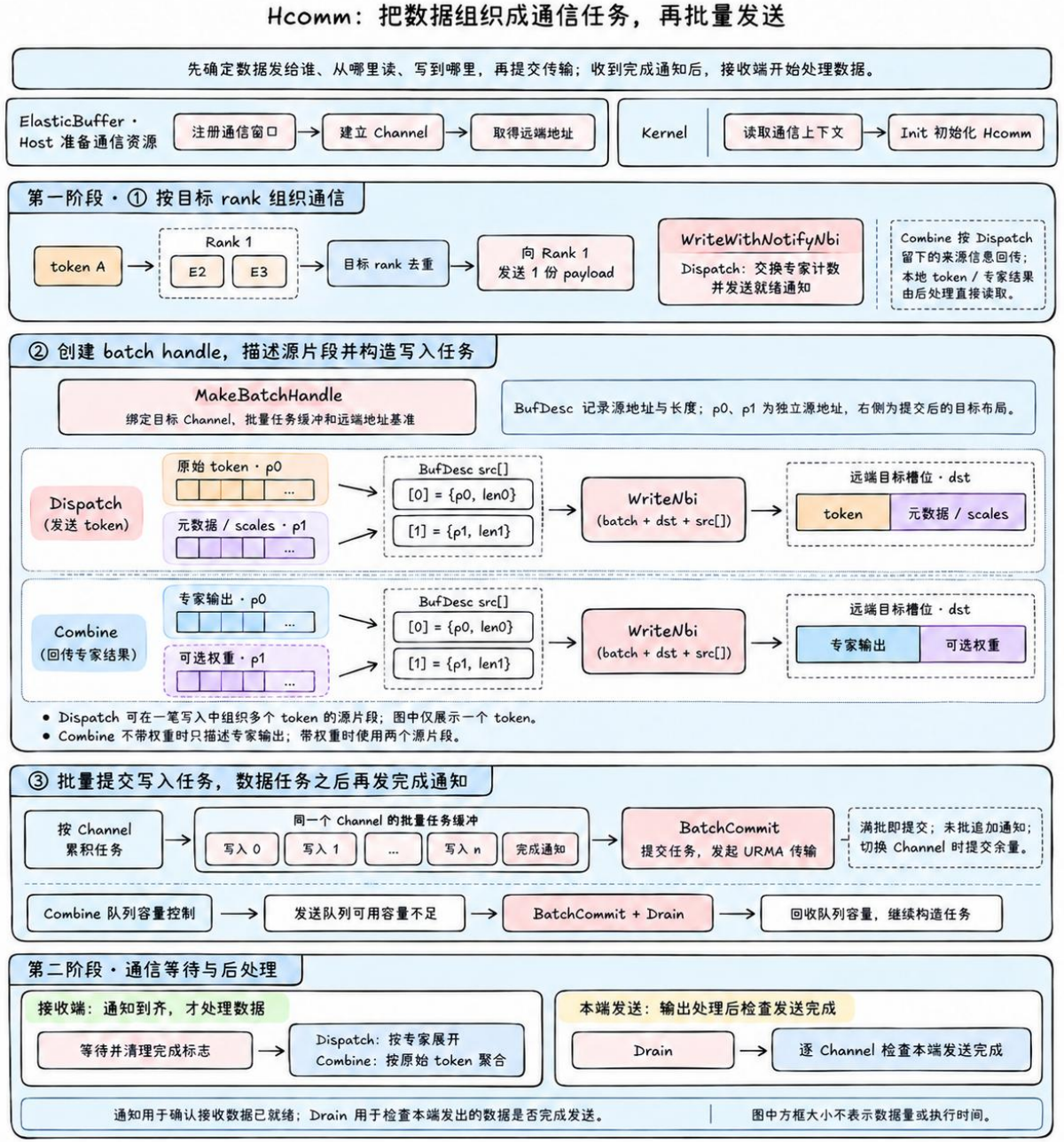
通过 Ascend C 通信 API 完成通信

ElasticBuffer 的 Host 侧准备并注册通信窗口，建立面向远端 rank 的 Channel，取得远端窗口地址，再将 Channel 句柄和窗口地址放入通信上下文。Kernel 从该上下文选取目标 Channel，并结合 token 路由计算本地源地址、远端目标地址与传输长度。

通信 API	当前实现中的使用方式
Hcomm::Init	使用 Kernel 分配的本地缓冲初始化 Hcomm 对象，供后续任务构造和提交使用。
Hcomm::WriteWithNotifyNbi	Dispatch 交换计数时，将目标 rank 上各专家的计数写入远端，同时发送携带该 rank 的 token 数及就绪标记的通知。
Hcomm::MakeBatchHandle	绑定目标 Channel、批量任务缓冲与远端地址基准，构造批量写入使用的 handle。
Hcomm::WriteNbi	向目标窗口写入数据或状态；payload 路径使用 batch handle 构造写入任务。Dispatch 通过 BufDesc 描述 token 与元数据两个源片段，Combine 描述专家输出及可选权重片段。
Hcomm::BatchCommit	将已构造的批量任务提交给 Channel；在批量容量达到阈值、需要切换 Channel 或收尾时提交剩余任务。
Hcomm::Drain	检查并等待 Channel 的发送完成。两个 Epilogue 在输出处理后逐 Channel 执行；Combine 发送阶段还会在发送队列可用容量不足时提交并等待，再继续发送。

以上仅说明当前实现中使用的一些通信 API 及其作用。

通过 Ascend C 通信 API 完成通信



围绕通信 API 做了以下组织：

- **按 rank 去重，再按专家展开。** Dispatch 的远端传输以去重后的目标 rank 为单位，避免同一 token 因命中同一 rank 的多个专家而重复发送 payload。
- **直接描述源数据片段。** 将原始 token 与暂存元数据组成源片段列表；通过 WriteNbi 写入目标槽位。
- **构造多笔任务后批量提交。** 两个发送算子均通过 batch handle 累积任务，使用 BatchCommit 提交，使任务构造与提交分开。Combine 还根据批量缓冲和发送队列的剩余容量控制提交与等待。
- **分别等待接收就绪和发送完成。** 发送端通过 Drain 等待本端发送完成。接收端通过强保序（odr=6）的状态信息判断数据是否到达。

性能数据

测试硬件为 **Ascend 950 DT**，PC16机型，16P。下表中的 num_tokens 均为每个 rank 的原始输入 token 数，Combine 通信数据类型均为bf16。时间单位为 **μs**，通信带宽按“通信数据量 ÷ URMA 通信时间”计算，单位为 **GB/s** 。

测试用例 h=7k，topk=6			dispatch性能					combine性能				
num_tokens	num_experts	dispatch_dtype	**总耗时**	前处理耗时	URMA通信耗时	按通信时间计算带宽	后处理耗时	**总耗时**	前处理耗时	URMA通信耗时	按通信时间计算带宽	后处理耗时
16384	256	fp8	**2078**	154	1277	539	646	**3604**	93	2945	467	566
16384	256	bf16	**3627**	194	2547	540	887	**3663**	92	2992	460	579
4096	384	fp8	**516**	79	259	664	178	**841**	32	653	527	156
4096	384	bf16	**879**	72	585	588	222	**836**	31	635	542	169

开源代码路径

以下路径均相位于 ops-transformer 仓库mc2目录，路径：https://gitcode.com/cann/ops-transformer/tree/master/mc2

内容	代码位置
ElasticBuffer 对外接口、两阶段提交、路由 handle 复用	mc2/common/torch_extension/elastic_buffer.py
通信内存注册、Channel 与远端窗口准备	mc2/common/torch_extension/csrc/elastic_buffer.cpp
Dispatch 路由去重、计数交换与批量发送	mc2/moe_ep_dispatch/op_kernel/moe_ep_dispatch.h
Dispatch 等待通知、按专家展开与输出元数据	mc2/moe_ep_dispatch_epilogue/op_kernel/moe_ep_dispatch_epilogue.h
Combine 回传地址计算、批量发送与 Channel 标志	mc2/moe_ep_combine/op_kernel/moe_ep_combine.h
Combine 本地直读、远端接收与 FP32 累加	mc2/moe_ep_combine_epilogue/op_kernel/moe_ep_combine_epilogue.h
两个 Epilogue 的发送完成检查	mc2/common/op_kernel/moe_ep_send_completion.h

目录

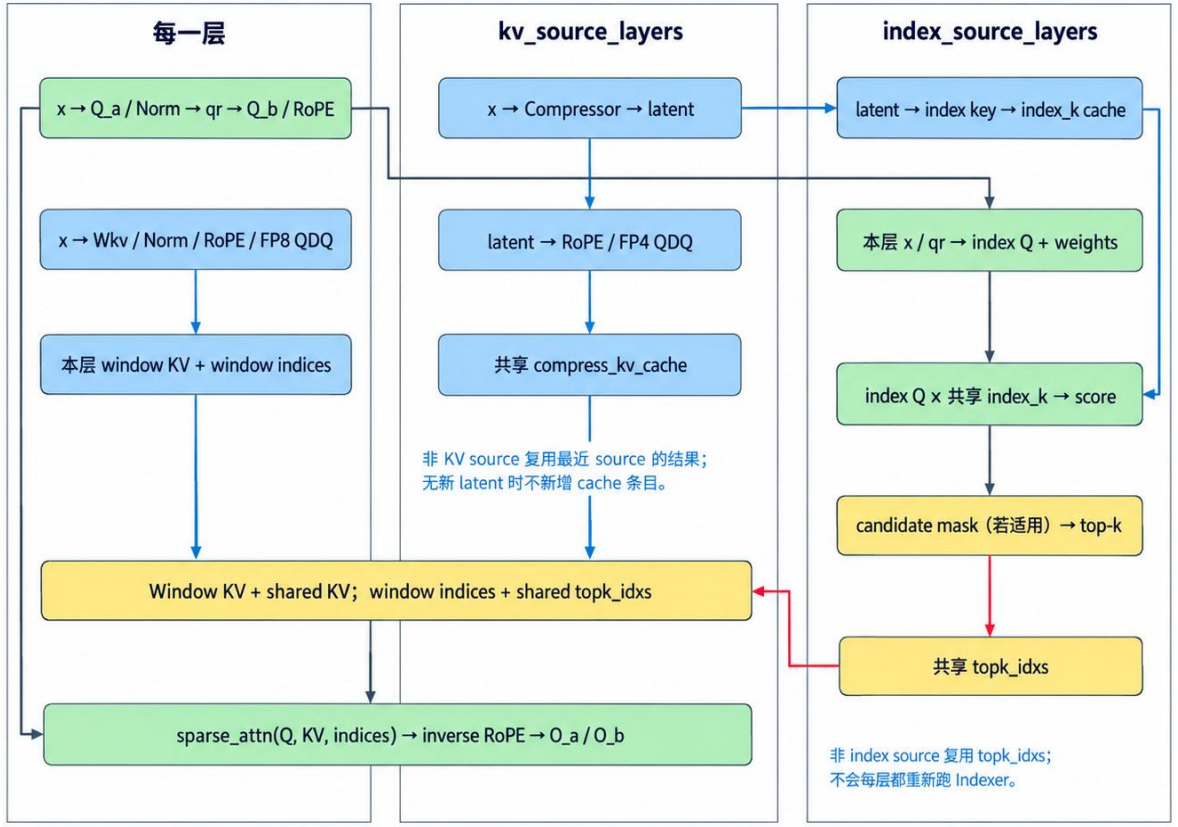
Part 1 自定义通信编程架构及API介绍

Part 2 基于通信API的高性能算子实践

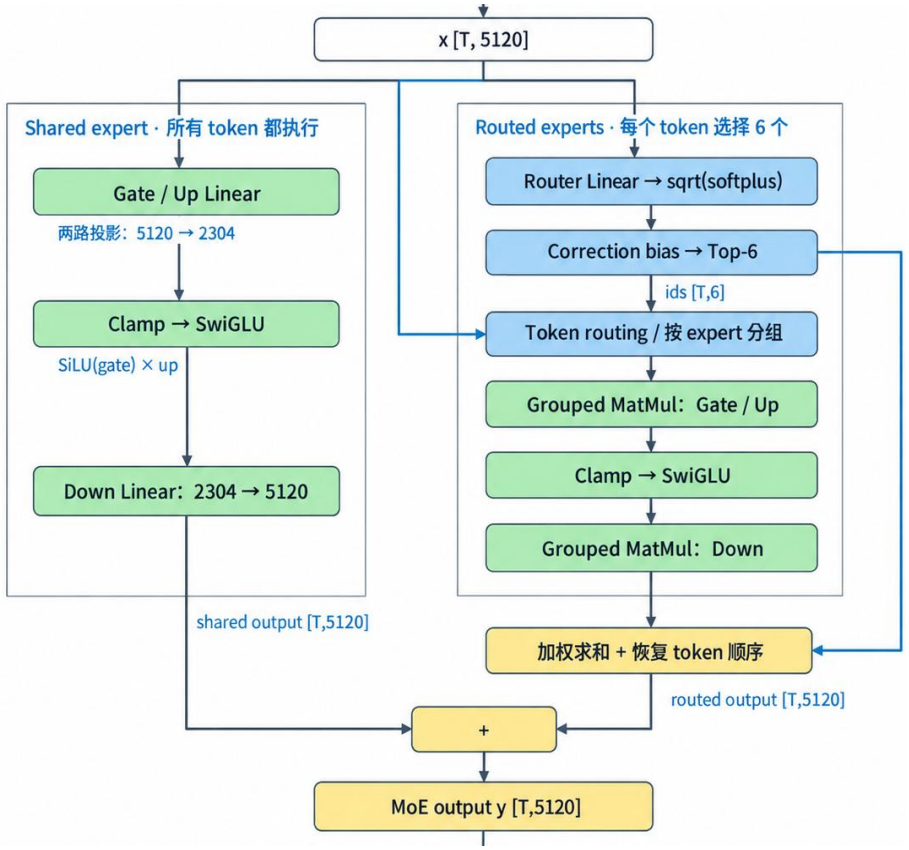
Part 3 单机极低时延推理参考实践

模型结构与极低时延场景

Attention模块



MOE模块



以DSv4.1网络结构为例，Attention模块使用Compressor压缩Cache和Indexer稀疏Attention，存在多条数据通路。

Moe模块有一个共享专家和多个路由专家。

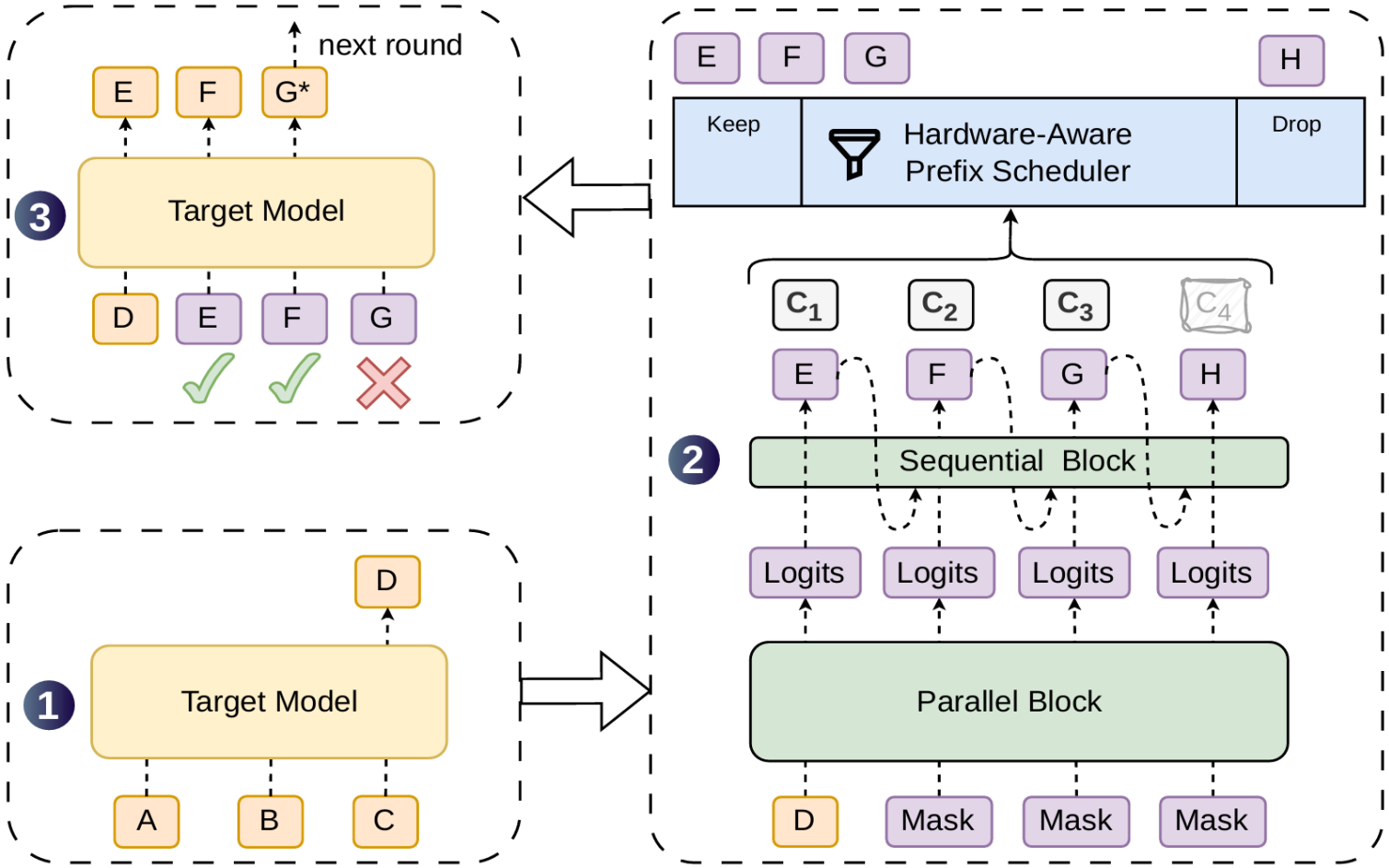
Decode常见高吞吐场景为Attention Dp + MOE Ep并行。

- Agent、语音交互、代码补全、工具调用，正在把推理推向极低时延；
- 针对整网单体机下的2ms级超低时延诉求和新的部署策略，我们基于DSv4系列网络打造了超低时延样例。

CANN

DSpark投机推理

DeepSeek-V4.1 Flash的DSpark方案支持Ascend 950系列，在一次草稿模型执行中生成一组Draft Token，由主模型批量Verify。与原生MTP逐步生成候选不同，DSpark使用多级Proposal Layer生成Draft Block，并通过Markov Head建模块内Token依赖。



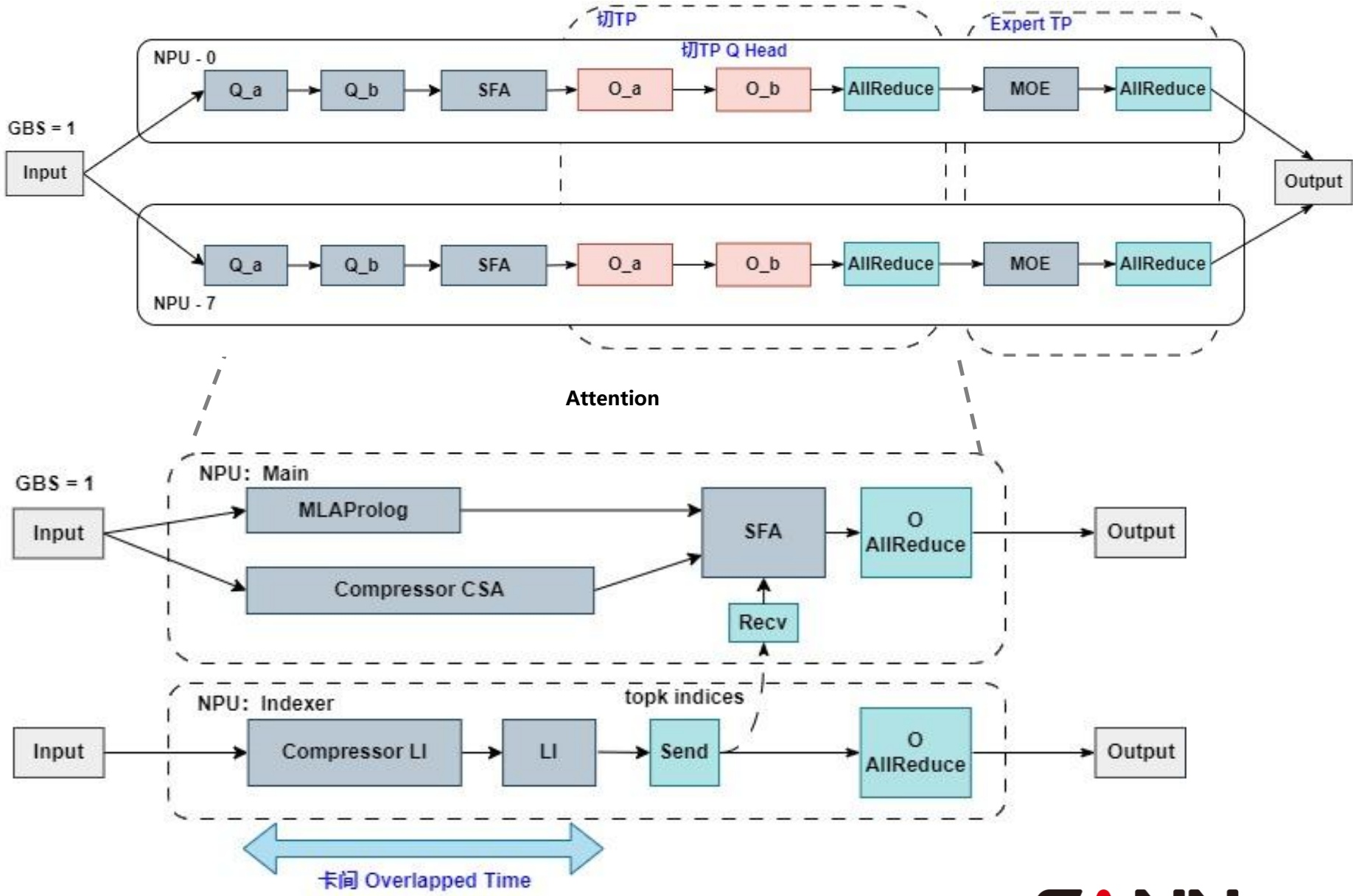
Attention部署策略

全局采用GBS=1的Tp部署

- Epilog采用tp部署，延续Q Head切分，节省一次通信。

Prolog叠加异构分卡部署

- 基于模型结构自带的prolog/li/compressor多条链路，将NPU拆分为异构Worker，部分执行 MLA、部分执行 Indexer；Indexer 反馈结果给 MLA。

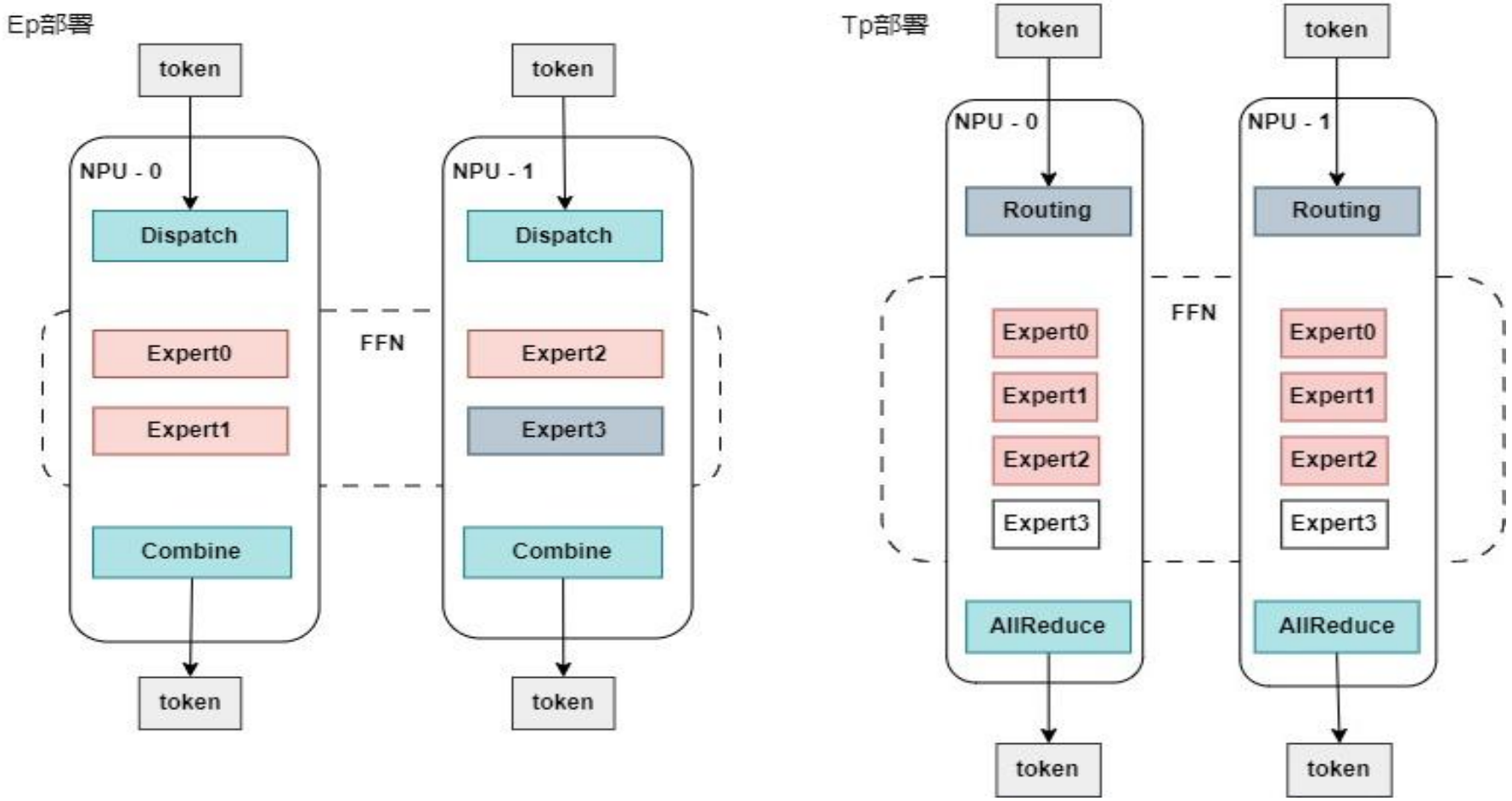


CANN

MOE部署策略

全局采用GBS=1的Tp部署：

- 延续前序模块的Tp切分：Attention模块的输出进入MoE模块后继续使用既有TP域，减少额外的Dispatch通信和Token重排；
- 负载均衡：TP部署保障各rank均有所有专家部分权重，避免负载集中于少数rank，并降低最大权重搬运量；
- 采用搬运亲和的Tp Shape切分。



- EP模式通过Dispatch/Combine通信在不同rank间交换Token，匹配Token与选择计算的Experts；
- TP模式在各rank保留所有专家的对应偏移分片，本地计算后通过all_reduce获得完整计算结果。低时延场景沿用前序模块的TP布局，以减少额外的Token分发。

CANN

极低时延优化

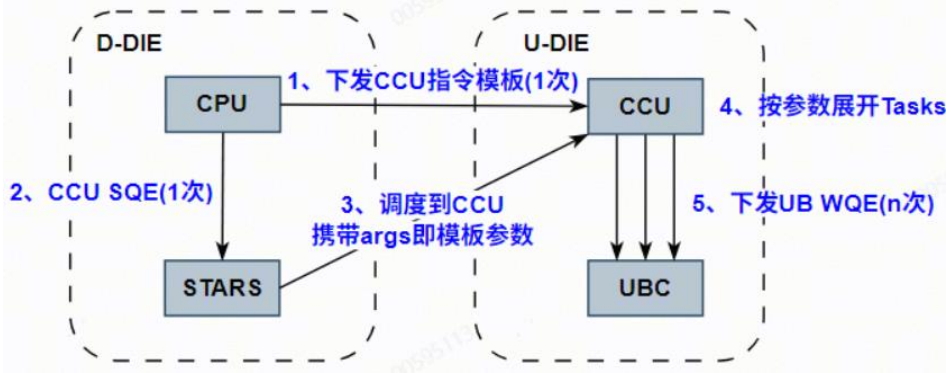
算子优化

- MOE模块
  - 稀疏 Grouplist 使用二维格式描述 expert_id 和 Token数，将有效专家排在前部，尾部保留无效或零 Token 项。在Token数量较少的极低时延场景，可以减少空专家的冗余遍历，降低Scalar开销；
  - 专家权重切分时，让不同rank成对上下对齐权重 Shape，使切分后的Shape对齐128等亲和Cache Line搬运的Shape，在搬运Bound的场景获取更低的计算时延。

切分方式	Rank0 GMM1 Shape	Rank0 GMM2 Shape	Rank1 GMM1 Shape	Rank1 GMM2 Shape
均匀切分	E=384, K=5120, N=288	E=384, K=144, N=5120	E=384, K=5120, N=288	E=384, K=144, N=5120
非均匀切分	E=384, K=5120, N=320	E=384, K=160, N=5120	E=384, K=5120, N=256	E=384, K=128, N=5120

框架优化

- Allreduce通信使用CCU展开，减少调度开销。



- 围绕Npugraph_ex图模式的框架优化：
  - npugraph_ex 捕获稳定的Decode图并在后续Step回放，避免Host逐算子下发。固定的算子拓扑、Cache地址和通信域有利于复用编译结果；
  - 通过事件依赖使能多流，将异构分卡的数据传递和共享专家阶段调度到独立 Stream，并行执行。

CANN

Benchmark & Future Plan

Benchmark

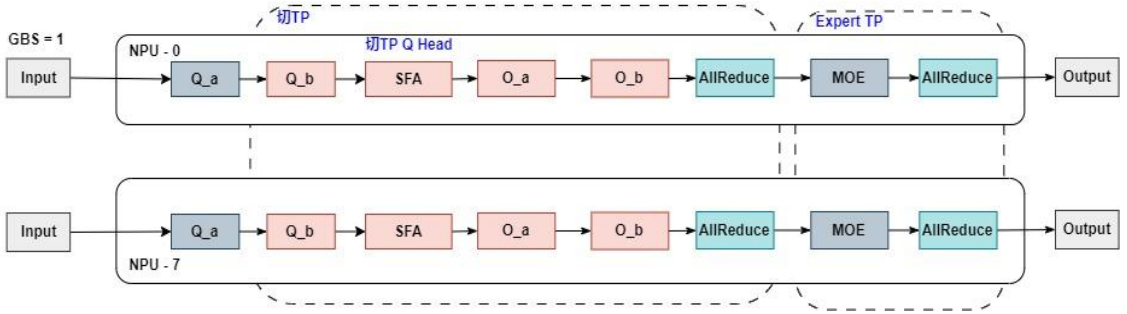
代码仓路径：
gitcode.com/cann/cann-recipes-infer/tree/master/docs/models/deepseek_v4_1

Global Batch Size	Chips	DSpark	DataType	Seq Length	TPOT (ms)
1	8	7	Hybrid MXFP8-MXFP4	8192	2.48

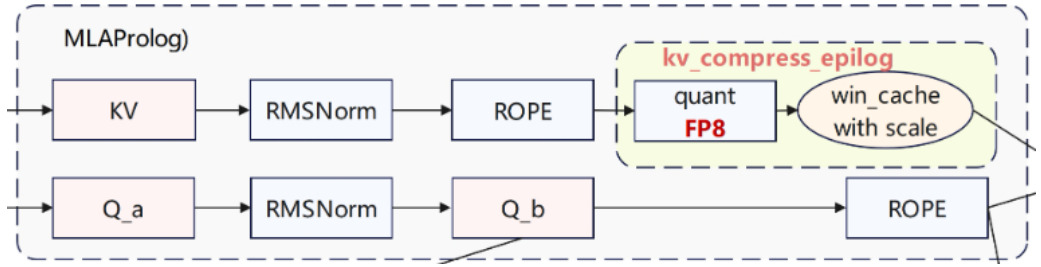
> 注：性能数据基于DSpark7采集，平均步长5.7，用户可按照数据集实际接受率自行折算benchmark性能。

Future Plan

部署：Q Head切分



算子：MLA融合算子



框架：SuperKernel 与预取

- 使能superkernel，降低算子间的调度开销；
- Prefetch权重预取，提前将访存bound算子的权重从HBM搬运到L2 Cache中。

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann